If one resource fails to commit (or times out), the transaction manager will have to retry this operation in a background thread until it either succeeds or reports the incident for manual intervention.

The one-phase commit (1PC) optimization

Because Java EE uses JTA transactions exclusively, the extra coordination overhead of the additional database roundtrip may hurt performance in a high-throughput application environment. When a transaction enlists only one resource adapter (designating a single resource manager), the transaction manager can skip the prepare phase, and either execute the commit or the rollback phase. With this optimization, the distributed transaction behaves similarly to how JDBC `Connection(s)` manage local transactions.

The `XAResource.commit(Xid xid, boolean onePhase`[a] method takes a `boolean` flag, which the transaction manager sets to *true* to hint the associated resource adapter to initiate the 1PC optimization.

[a] https://docs.oracle.com/javaee/7/api/javax/transaction/xa/XAResource.html#commit-javax.transaction.xa.Xid-boolean-

6.6.2 Declarative transactions

Transaction boundaries are usually associated with a Service layer, which uses one or more DAO to fulfill the business logic. The transaction *propagates* from one component to the other within the service-layer transaction boundaries.

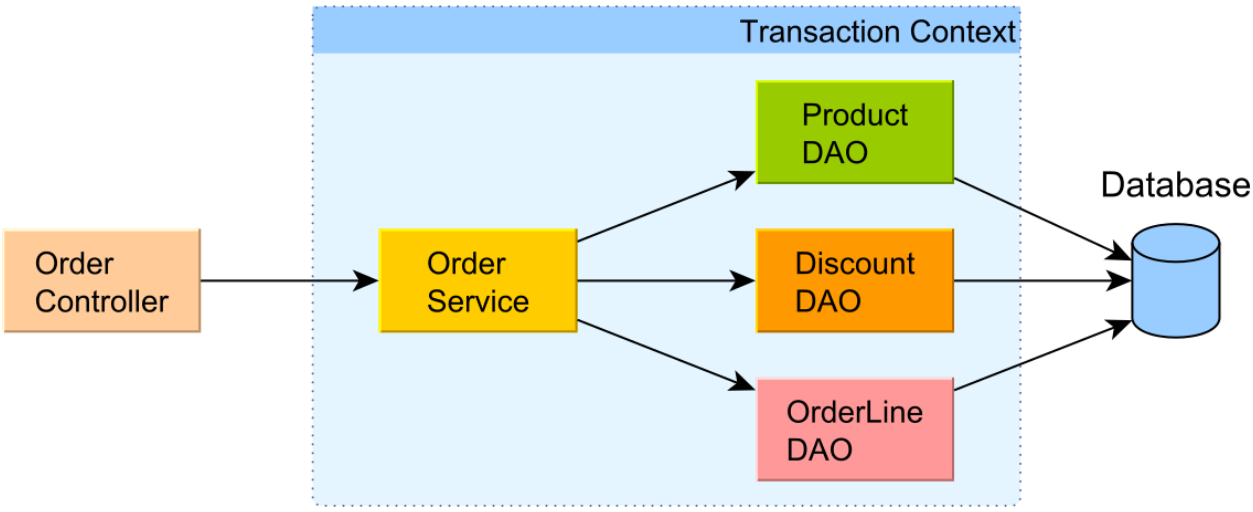

Figure 6.13: Transaction boundaries

The declarative transaction model is supported by both Java EE and Spring. Transaction boundaries are controlled through similar propagation strategies, which define how boundaries are inherited or disrupted at the borderline between the outermost component (in the current call stack) and the current one (waiting to be invoked).

Propagation

To configure the transaction propagation strategy for EJB components, Java EE defines the `@TransactionAttribute`[a] annotation. Since Java EE 7, even non-EJB components can now be enrolled in a transactional context if they are augmented with the `@Transactional`[b] annotation.

In Spring, transaction propagation (like any other transaction properties) is configurable via the `@Transactional`[c] annotation.

[a] http://docs.oracle.com/javaee/7/api/javax/ejb/TransactionAttribute.html
[b] http://docs.oracle.com/javaee/7/api/javax/transaction/Transactional.html
[c] https://docs.spring.io/spring/docs/current/javadoc-api/org/springframework/transaction/annotation/Transactional.html#propagation--

Table 6.6: Transaction propagation strategies

Propagation	Java EE	Spring	Description
REQUIRED	Yes	Yes	This is the default propagation strategy, and it only starts a transaction unless the current thread is not already associated with a transaction context
REQUIRES_NEW	Yes	Yes	Any currently running transaction context is suspended and replaced by a new transaction
SUPPORTS	Yes	Yes	If the current thread already runs inside a transaction, this method will use it. Otherwise, it executes outside of a transaction context
NOT_SUPPORTED	Yes	Yes	Any currently running transaction context is suspended, and the current method is run outside of a transaction context
MANDATORY	Yes	Yes	The current method runs only if the current thread is already associated with a transaction context
NESTED	No	Yes	The current method is executed within a nested transaction if the current thread is already associated with a transaction. Otherwise, a new transaction is started.
NEVER	Yes	Yes	The current method must always run outside of a transaction context, and, if the current thread is associated with a transaction, an exception will be thrown.

Declarative exception handling

Since the transaction logic wraps around the underlying service and data access logic call chain, the exception handling must also be configured declaratively. By default, both Java EE and Spring roll back on system exceptions (any `RuntimeException`) and commit on application exceptions (checked exceptions).

In Java EE, the rollback policy can be customized using the `@ApplicationException`[a] annotation.

Spring allows each transaction to customize the rolling back policy[b] by listing the exception types triggering a transaction failure.

[a] http://docs.oracle.com/javaee/7/api/javax/ejb/ApplicationException.html
[b] https://docs.spring.io/spring/docs/current/javadoc-api/org/springframework/transaction/annotation/Transactional.html#rollbackFor--

Declarative read-only transactions

Java EE does not support read-only transactions to be marked declaratively.

Spring offers the transactional read-only attribute[a], which can propagate to the underlying JPA provider (to optimize the `EntityManager` flushing mechanism) and to the current associated JDBC `Connection`.

[a] https://docs.spring.io/spring/docs/current/javadoc-api/org/springframework/transaction/annotation/Transactional.html#readOnly--

Declarative isolation levels

The Java EE does not offer support for configurable isolation levels, so it is up to the underlying `DataSource` to define it for all database connections.

Spring supports transaction-level isolation levels[a] when using the `JpaTransactionManager`[b]. For JTA transactions, the `JtaTransactionManager`[c] follows the Java EE standard and disallows overriding the default isolation level. As a workaround, the Spring framework provides extension points, so the application developer can customize the default behavior and implement a mechanism to set isolation levels on a transaction basis.

[a] https://docs.spring.io/spring/docs/current/javadoc-api/org/springframework/transaction/annotation/Transactional.html#isolation--
[b] https://docs.spring.io/spring/docs/current/javadoc-api/org/springframework/orm/jpa/JpaTransactionManager.html
[c] https://docs.spring.io/spring/docs/current/javadoc-api/org/springframework/transaction/jta/JtaTransactionManager.html

6.7 Application-level transactions

So far, the book focused on database transactions to enforce ACID properties. However, from the application perspective, a business workflow might span over multiple physical database transactions, in which case the database ACID guarantees are not sufficient anymore. A logical transaction may be composed of multiple web requests, including user think time, for which reason it can be visualized as a *long conversation*.

In the following example, *Alice* and a background process are concurrently modifying the same database record.

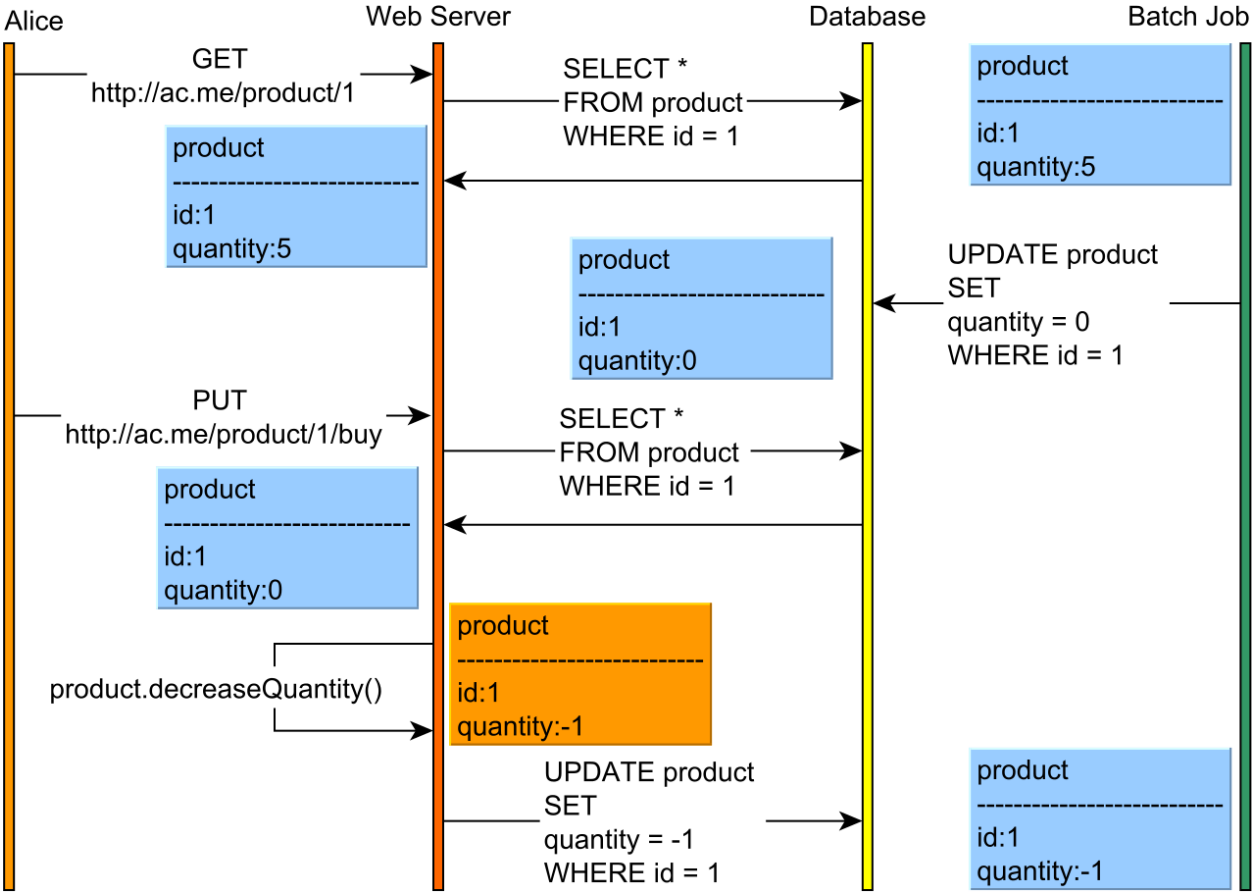

Figure 6.14: Stateless conversation losing updates

Because *Alice* logical transaction encloses two separate web requests, each one associated with a separate database transaction, without an additional concurrency control mechanism, even the strongest isolation level cannot prevent the lost update phenomena.

Spanning a database transaction over multiple web requests is prohibitive since locks would be held during user think time, therefore hurting scalability. Even with MVCC, the cost of maintaining previous versions (that can lead to a large version graph) can escalate and affect both performance and concurrency. Application-level transactions require application-level concurrency control mechanisms.

HTTP is stateless by nature and, for very good reasons, stateless applications are easier to scale than stateful ones. However, application-level transactions cannot be stateless, as otherwise newer requests would not continue from where the previous request was left. Preserving state across multiple web requests allows building a conversational context, providing application-level repeatable reads guarantees.

In the next diagram, *Alice* uses a stateful conversational context, but, in the absence of a record versioning system, it is still possible to lose updates.

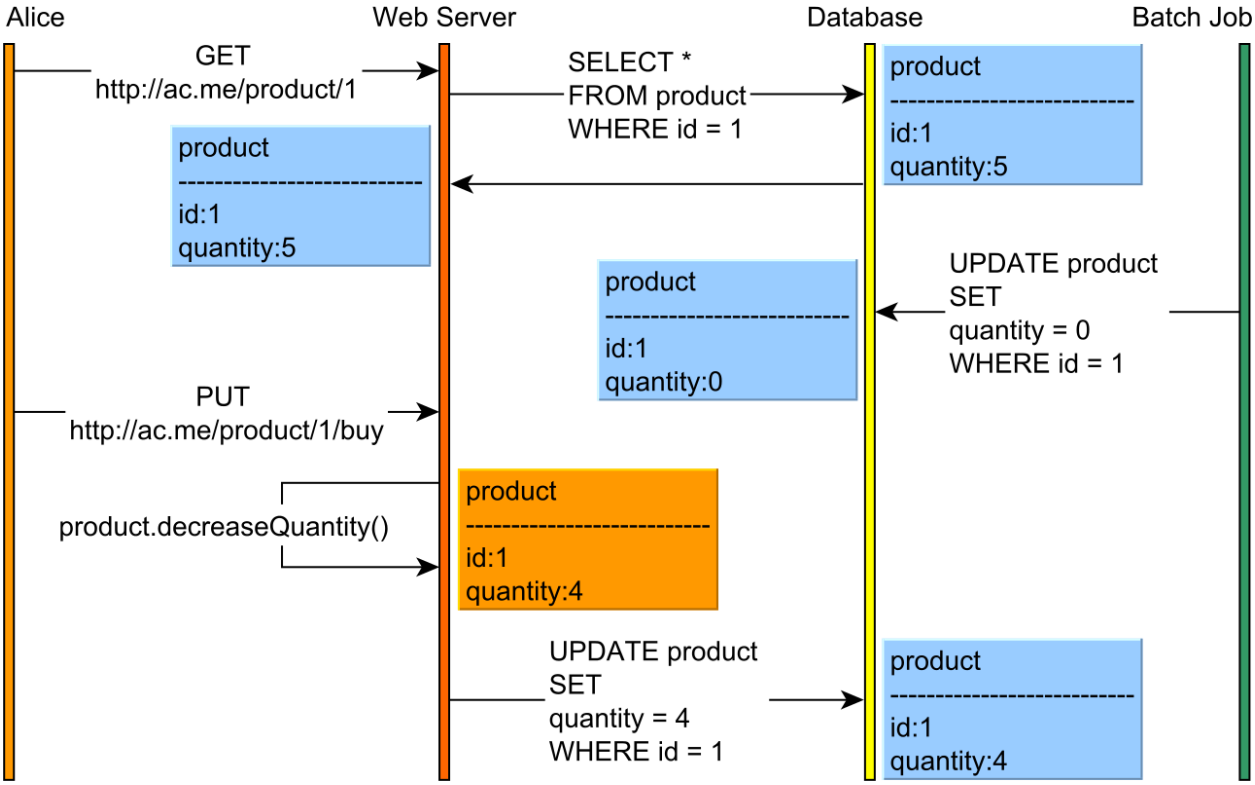

Figure 6.15: Stateful conversation losing updates

Without *Alice* to notice, the batch process resets the product quantity. Thinking the product version has not changed, *Alice* attempts to purchase one item which decreases the previous product quantity by one. In the end, *Alice* has simply overwritten the batch processor modification, and data integrity has been compromised.

So the application-level repeatable reads are not self-sufficient (this argument is true for database isolation levels as well). To prevent lost updates, a concurrency control mechanism becomes mandatory.